

Sundbusserne ATVM

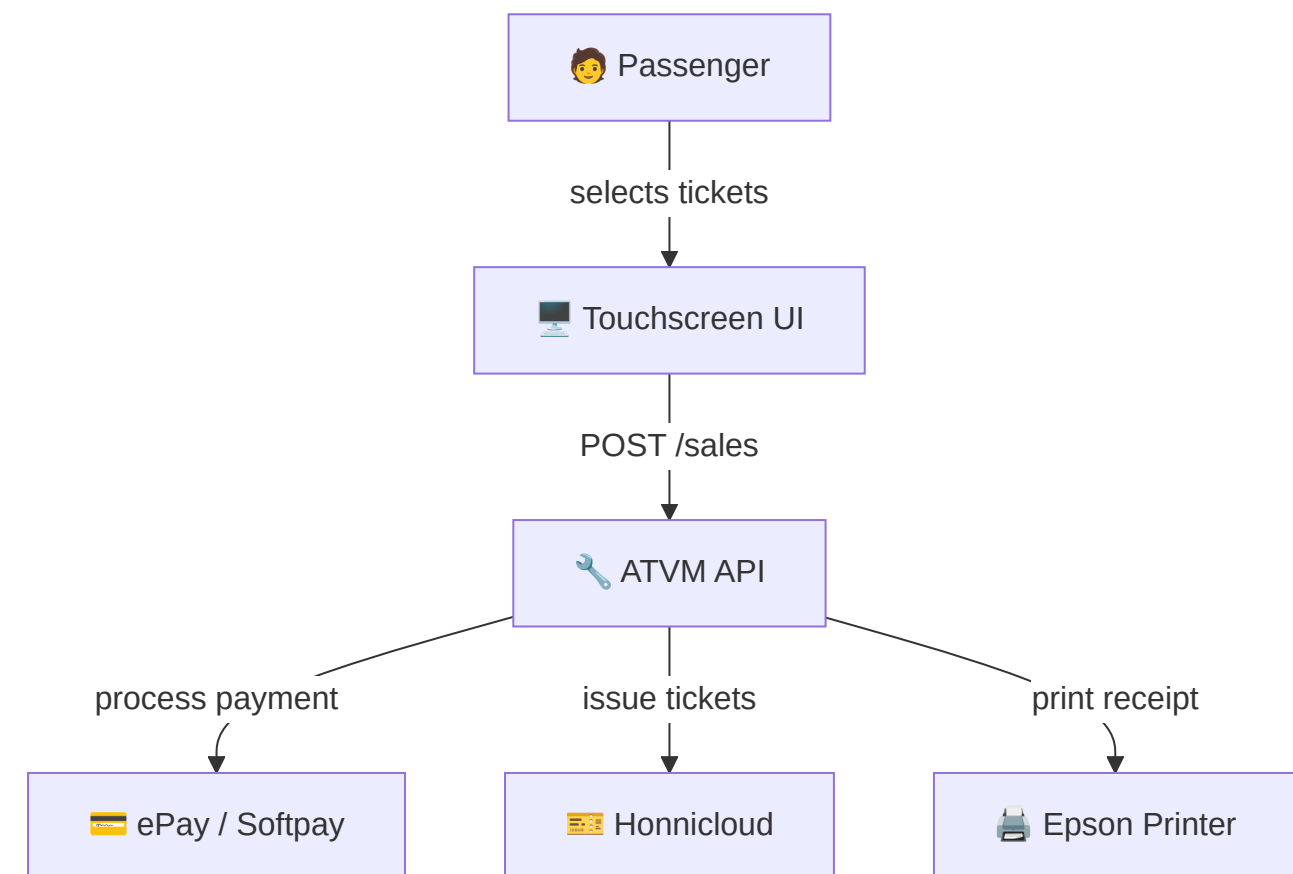
Automatic Ticket Vending Machine

2026 Honnicloud

ATVM — Automatic Ticket Vending Machine

Executive Overview

ATVM is a **self-service ticket vending system** that runs on physical machines at transport hubs (bus stations, train stations, etc.). Passengers select tickets on a touchscreen, pay using one of several methods, and receive printed tickets — all without staff assistance.



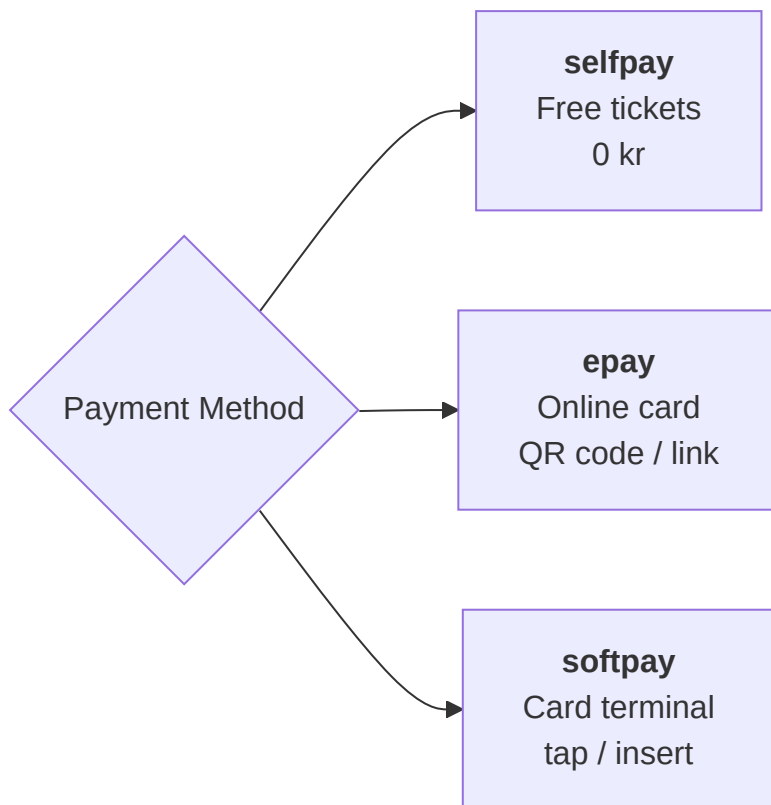
How It Works — Step by Step

Step 1: The Passenger Selects Tickets

Each vending machine displays one or more **pages** on its touchscreen. Pages contain **tiles** — each tile represents a ticket type (e.g., “Adult Single”, “Child Return”, “Bicycle”). Tiles show prices, names in multiple languages (Danish, English, Swedish), and optional images.

Step 2: The Passenger Pays

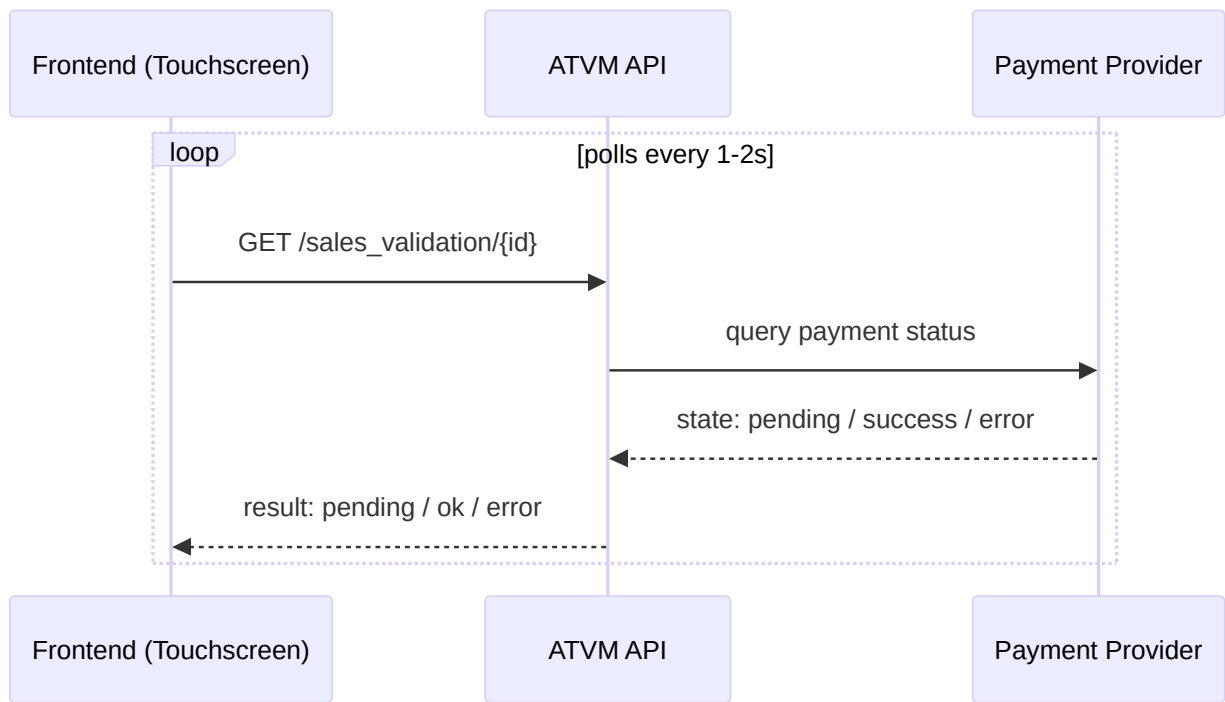
When the passenger presses “Pay”, the system creates a **sale** and uses one of three payment methods:



Payment Type	Use Case	How It Works
selfpay	Free tickets (children under 6, staff)	No external call. Tickets issued immediately.
epay	Online payments	Creates a payment session. Passenger scans a QR code or follows a link to pay on their phone.
softpay	Physical card payments	Activates the attached card terminal. Passenger taps or inserts their card.

Step 3: Payment Is Confirmed

After payment, the system checks the result by polling a validation endpoint:

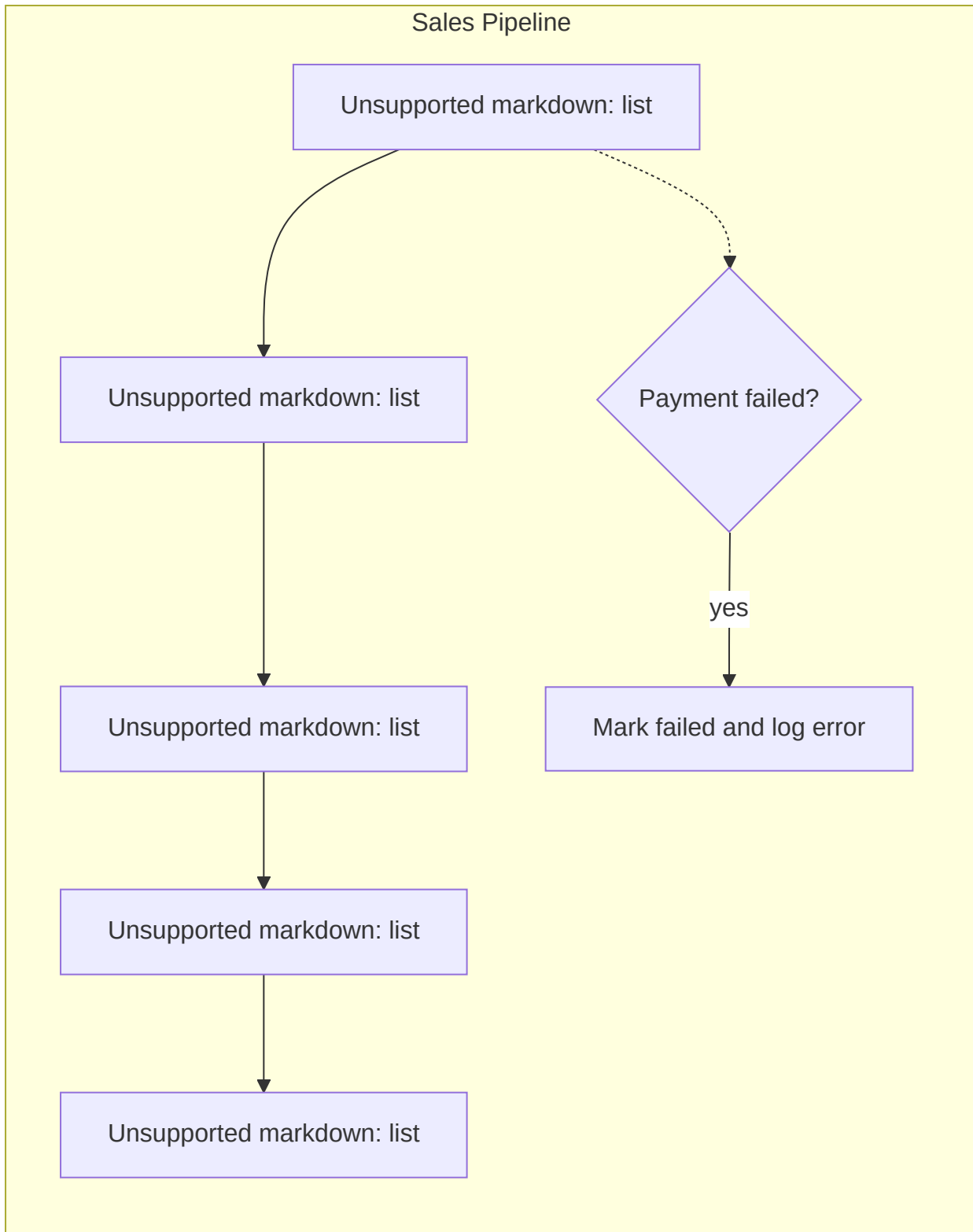


The validation returns one of:

- **pending** — payment still in progress, keep waiting
- **ok** — payment succeeded, proceed to issue tickets
- **error** — payment failed, show error message
- **cancelled** — passenger cancelled

Step 4: Tickets Are Issued

Once payment is confirmed as **ok**, a background **pipeline** takes over:



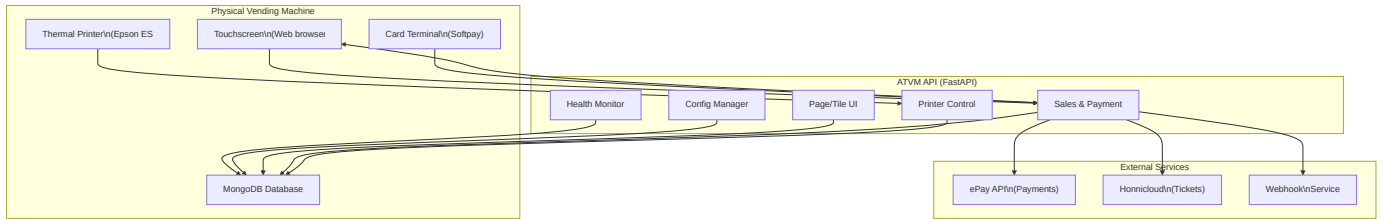
The pipeline calls the **Honnicloud ticketing API** to issue one digital ticket per purchased item. Each ticket gets a unique ID, validity dates, and a QR/barcode.

Step 5: Tickets Are Printed

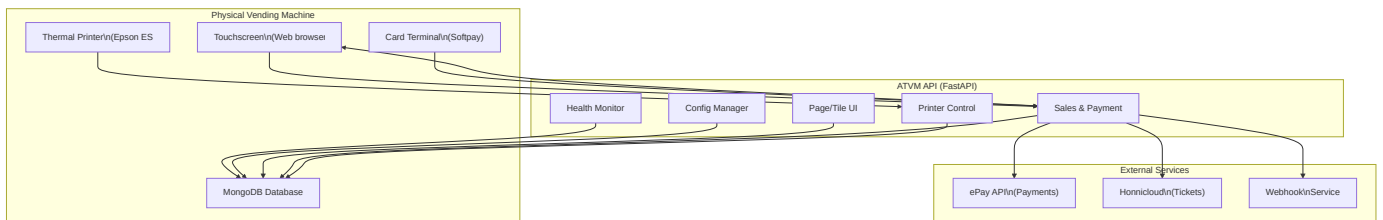
The printed receipt includes:

- All issued tickets with their IDs
- Ticket names, valid-until dates

- Total amount paid
- Receipt number (bon_id)



System Architecture



Key Components

1. Configuration Management

Each machine has its own configuration — language, currency, payment provider credentials, and terminal IDs. This is managed through `/config/basesetup`.

2. Page & Tile Management

The touchscreen UI is built from **pages** containing **tiles**. Each tile represents a ticket type with pricing, multi-language labels, and optional images. Pages are configured per-machine through `/config/page`.

3. Sales & Payment

The core transaction flow:

1. Create a sale (`POST /sales`) with a payment method
2. Poll for payment status (`GET /sales_validation/{id}`)
3. When payment succeeds, the background pipeline issues tickets

4. Print the receipt (`POST /printer/receipt/{id}`)

4. Printer Control

Manages the Epson thermal printer — checking status (online, paper, cover), printing tickets, and printing receipts.

5. Health Monitoring

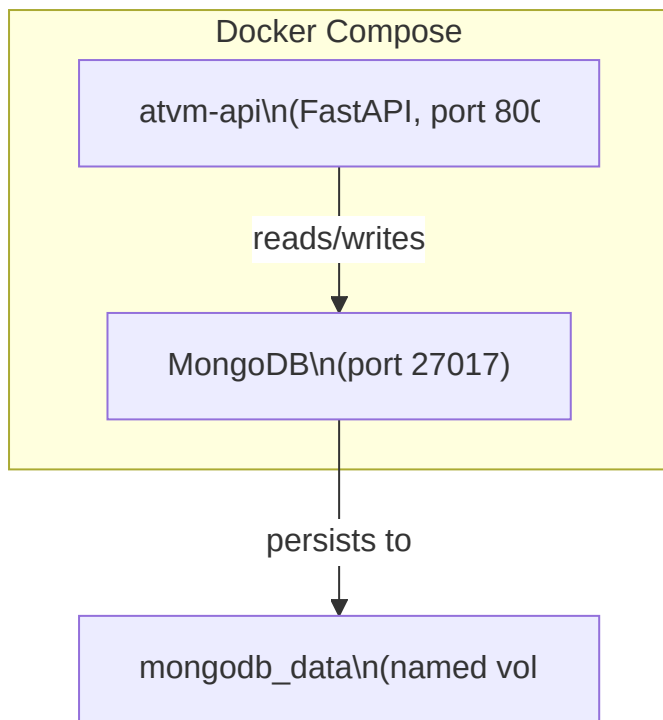
The `/health` endpoint checks connectivity to all dependent services (MongoDB, Honnicloud API) and reports overall system health.

Technology Stack

Layer	Technology
API Framework	FastAPI (Python 3.11)
Database	MongoDB 7 (Motor async driver)
Deployment	Docker + Docker Compose
Payment	ePay (online) / Softpay (terminal)
Ticketing	Honnicloud API
Printing	Epson ESC/POS (python-escpos)
Package Manager	uv

Deployment

Each vending machine runs a Docker Compose stack with two containers:



The API is stateless — all state lives in MongoDB. Machines are identified by unique machine IDs with Bearer token authentication.

Multi-Machine Support

A single ATVM deployment can manage multiple machines, each with:

- Unique configuration (currency, language, payment provider)
- Custom page layouts and ticket offerings
- Independent printer settings
- Separate transaction history

Machines communicate with the API over the local network or internet, authenticated by machine-specific keys.

ATVM API Deployment Guide

This guide covers deploying the ATVM API using Docker.

Prerequisites

- Docker Engine 20.10+
- Docker Compose 2.0+
- Git (for cloning the repository)

Quick Start

1. Clone the repository:

```
git clone <repository-url>  
cd atvm
```

2. Copy the environment file:

```
cp .env.example .env
```

3. Start the application:

```
docker-compose up -d
```

4. Verify the application is running:

```
curl http://localhost:8000/health
```

Configuration

Environment Variables

Edit the `.env` file to configure the application:

```
# MongoDB configuration
MONGODB_URL=mongodb://atvm:atvmsecret@mongodb:27017/atvm?authSource=admin
MONGODB_DB=atvm
MONGO_USERNAME=atvm
MONGO_PASSWORD=atvmsecret

# Honnicloud ticketing API
HONNICLOUD_API_KEY=<your-api-key>

# Logging
LOG_LEVEL=INFO

# Server settings
PORT=8000
```

Database

MongoDB is deployed as part of the Docker Compose stack. Data is stored in the `mongodb_data` named Docker volume, which persists across container restarts.

Collections

Collection	Purpose
<code>machine_configs</code>	Per-machine base configuration (language, currency, epay URLs, ...)
<code>pages</code>	Per-machine page/tile layout for the UI
<code>sales</code>	Sales records including honnicloud ticket id and payment result
<code>printer_configs</code>	Per-machine Epson printer settings (ip, port, header/footer)

Printer Configuration

Printer settings are no longer read from `config.ini`. Seed them via the API after initial deployment:

```
curl -X POST http://localhost:8000/config/printer \
  -H "Authorization: Bearer <machine_key>" \
  -H "Content-Type: application/json" \
  -d '{"name":"TM-T20II","ip":"192.168.200.197","port":9100,"logoheader":"YOUR COMPANY","logofooter":"Company 2026"}'
```

```
docker exec atvm-mongodb mongodump \
  --uri "mongodb://atvm:atvmsecret@localhost:27017/atvm?authSource=admin" \
  --out /tmp/atvm-backup
```

Authentication

Machine keys are defined in `atvm/auth.py`. To modify authentication:

1. Build a custom Docker image with updated `auth.py`
2. Or mount a custom `auth.py` file as a volume

Default machine keys:

- `helsingør_01`: `hedensted2026`
- `helsingborg_01`: `korning2026`

Docker Commands

Building the Image

```
docker build -t atvm-api .
```

Running with Docker Compose

```
# Start in detached mode
docker-compose up -d
```

```
# View logs
docker-compose logs -f
```

```
# Stop the application
docker-compose down
```

```
# Stop and remove volumes
docker-compose down -v
```

```
# Rebuild and restart
docker-compose up -d --build
```

Running with Docker Directly

```
# Run container (MongoDB must already be running)
docker run -d \
  -p 8000:8000 \
  -v $(pwd)/logs:/app/logs \
  -e MONGODB_URL=mongodb://atvm:atvmsecret@mongodb:27017/atvm?authSource=admin \
  --name atvm-api \
  atvm-api
```

Production Considerations

1. Security

- Change default machine keys in production
- Use HTTPS with reverse proxy (nginx, Traefik)
- Set appropriate file permissions
- Regularly update Docker images

2. Database

- MongoDB data is in the `mongodb_data` Docker volume — back it up regularly
- Consider MongoDB Atlas or a replica set for high availability
- Change `MONGO_PASSWORD` from the default before going to production

3. Monitoring

- Application logs are stored in `logs/` directory
- Health endpoint: `GET /health`
- Consider adding metrics endpoint

4. Scaling

- The API is stateless — scale horizontally behind a load balancer
- MongoDB handles concurrent connections from multiple API instances

5. Updates

To update the application:

```
# Pull latest code
git pull

# Rebuild and restart
docker compose up -d --build
```

Troubleshooting

Common Issues

1. Port already in use

```
# Change PORT in .env file
PORT=8001
```

2. Permission denied on logs directory

```
# Fix permissions for Docker user (UID 1000)
sudo chown -R 1000:1000 logs
```

3. API cannot connect to MongoDB

- Ensure MongoDB container is healthy: `docker compose ps`
- Verify `MONGODB_URL` in `.env` matches the credentials in `MONGO_USERNAME / MONGO_PASSWORD`
- Check MongoDB logs: `docker compose logs mongodb`

4. MongoDB authentication failure

- The `MONGO_INITDB_ROOT_USERNAME / PASSWORD` only take effect on first volume initialisation
- To reset: `docker compose down -v && docker compose up -d`
- Review API logs: `docker compose logs atvm-api`

View application logs

```
docker-compose logs atvm-api
```

Checking Logs

```
# Docker Compose logs
docker-compose logs -f atvm-api

# Docker logs
docker logs atvm-api

# Application logs (mounted volume)
tail -f logs/atvm.log
```

API Endpoints

Health Check (No authentication)

```
GET /health
```

Configuration (Requires authentication)

```
GET    /config/basesetup
POST   /config/basesetup
```

Pages (Requires authentication)

```
GET    /config/page?page=1
POST   /config/page
```

Authentication

Add header to authenticated requests:

```
Authorization: Bearer <machine_key>
```

Support

For issues or questions:

1. Check the logs: `docker-compose logs atvm-api`
2. Review this deployment guide
3. Check the README.md for application details

Ticket Data Import Tool

This tool imports ticket data from the Honnicloud API (`ticket.honnicloud.com`) to update ATVM tile configurations in MongoDB.

Purpose

The tool:

1. Connects to MongoDB and fetches all pages from the `pages` collection
2. Extracts all tiles with `ticket_type_id` values
3. Fetches the product catalog from Honnicloud API
4. For each tile, finds the matching product by `ref = ticket_type_id`
5. Updates tile data with:
 - `price` from product price
 - `ticket_name` from product name
 - `text_by_language` from product `localized_name`
6. Prints warnings for `ticket_type_id` values not found in the API
7. Saves updated pages back to MongoDB

Prerequisites

1. **MongoDB must be running** (either locally or in Docker)
2. **Environment variables** must be set (see `.env` file):
 - `HONNICLOUD_API_KEY` - API key for Honnicloud
 - `HONNICLOUD_TICKET_URL` - API base URL (default: `https://api.honnicloud.com`)
 - `MONGODB_URL` - MongoDB connection string
 - `MONGODB_DB` - Database name (default: `atvm`)

Usage

1. Running in Docker Environment

When deployed with Docker Compose:

```
# Ensure Docker containers are running
docker-compose up -d

# Run the import tool inside the ATVM container
docker-compose exec atvm python /app/import_ticket_data.py
```

2. Running Locally (Development)

```
# Set environment variables (or use .env file)
export HONNICLOUD_API_KEY="your_api_key_here"
export MONGODB_URL="mongodb://localhost:27017"

# Run the tool
python import_ticket_data.py
```

3. Using .env File

The tool automatically reads from the `.env` file in the current directory:

```
# Make sure .env file exists with required variables
python import_ticket_data.py
```

Expected Output

```
2025-03-26 11:30:00 | INFO      | Starting ticket data import tool
2025-03-26 11:30:00 | INFO      | Successfully connected to MongoDB
2025-03-26 11:30:01 | INFO      | Fetched 150 products from Honnicloud API
2025-03-26 11:30:01 | INFO      | Created product map with 145 entries
2025-03-26 11:30:01 | INFO      | Found 5 pages to process
2025-03-26 11:30:01 | INFO      | Updated page 1 for machine helsingør_01
2025-03-26 11:30:01 | INFO      | Updated page 2 for machine helsingør_01
```

```
=====
IMPORT SUMMARY
=====
```

```
Total tiles processed: 45
Pages updated: 5
Ticket type IDs not found in API (2 unique):
- 111010
- 111011
Import completed successfully
```

Data Flow

1. **Input:** MongoDB `pages` collection with tiles containing `ticket_type_id`
2. **API Call:** Fetch products from `GET /products` endpoint
3. **Matching:** Find products where `product.ref = tile.ticket_type_id`
4. **Updates:**
 - `tile.price ← product.price`
 - `tile.ticket_name ← product.name`
 - `tile.text_by_language ← product.localized_name`
5. **Output:** Updated `pages` collection in MongoDB

Error Handling

- **MongoDB connection failure:** Tool exits with clear error message
- **API connection failure:** Tool exits if no products can be fetched
- **Missing ticket_type_id:** Tiles without `ticket_type_id` are skipped
- **Unmatched ticket_type_id:** Warning printed, tile remains unchanged

Testing

A test script is available to verify the logic without actual API/MongoDB connections:

```
python test_import_tool.py
```

Integration with ATVM System

This tool is designed to work with the existing ATVM system:

- Uses the same MongoDB connection settings as the main application
- Leverages the existing `honncloud.py` module patterns
- Updates tile data in the format expected by the ATVM frontend
- Can be run periodically to keep ticket data synchronized

Notes

- The tool only updates existing tiles - it does not create new tiles or pages

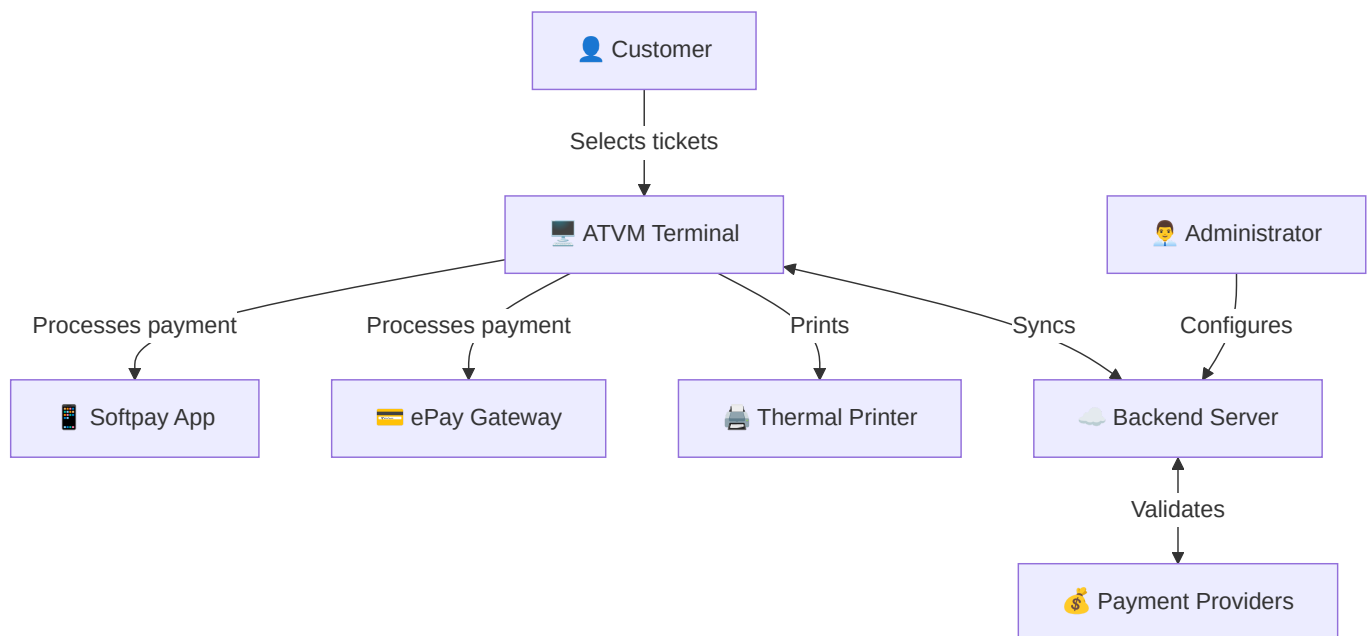
- Empty `ticket_type_id` values are skipped
- Price conversion to float handles various numeric formats
- `text_by_language` is completely replaced with data from `localized_name`
- Run time depends on number of pages/tiles and API response time

ATVM — Automated Ticket Vending Machine

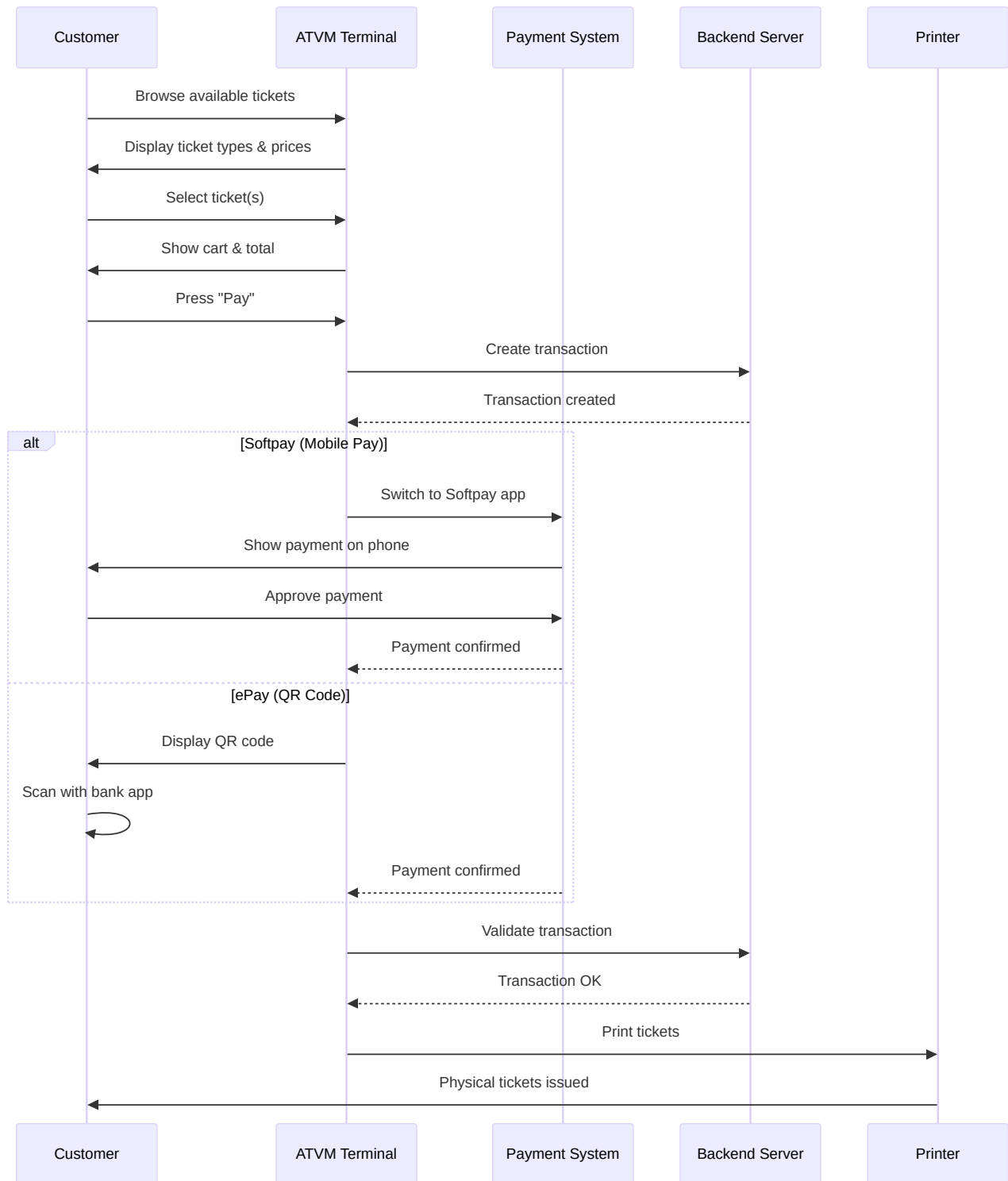
Executive Overview

The **Automated Ticket Vending Machine (ATVM)** is a self-service ticketing system designed for **Sundbusserne**, enabling passengers to purchase tickets independently through an intuitive touch-screen terminal. The system runs on a Sunmi Android device with an integrated thermal printer, providing a seamless end-to-end experience from ticket selection to physical receipt — without requiring staff intervention.

1. System At a Glance

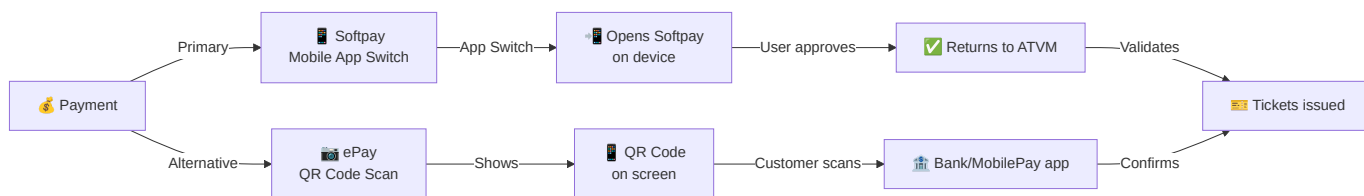


2. The Customer Journey



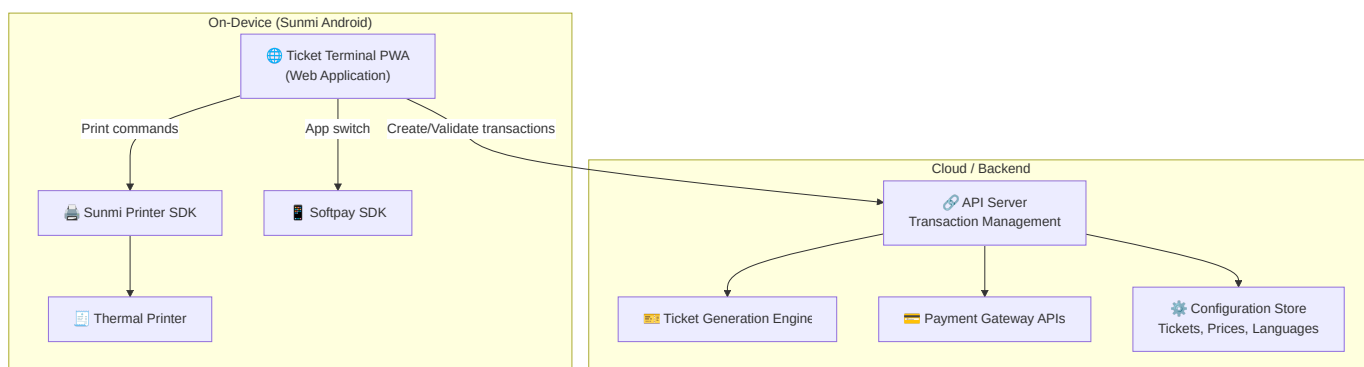
3. Payment Methods

The terminal supports two payment methods to accommodate different customer preferences:



Method	How It Works	Best For
Softpay	The terminal switches to the Softpay app on the same device. The customer approves the payment, then returns to the terminal automatically.	Customers with Softpay/MobilePay installed
ePay (Alt.)	The terminal displays a QR code. The customer scans it with their banking app to complete payment.	Customers who prefer scanning over app switching

4. System Architecture



5. Key Components

ATM Terminal (Frontend)

A Progressive Web App (PWA) running in fullscreen mode on a Sunmi Android device. It provides:

- **Ticket browsing** — Displays available ticket types with prices
- **Shopping cart** — Add/remove tickets, see running total
- **Checkout** — Select payment method and process payment
- **Multi-language support** — Switch between Danish, English, German, and Swedish
- **Status monitoring** — Printer health, server connectivity, and transaction status

Backend Server

The central brain of the system, responsible for:

- **Ticket catalog** — Managing available ticket types, prices, and validity periods
- **Transaction processing** — Creating and validating payment transactions
- **Ticket issuance** — Generating unique ticket IDs and QR codes
- **Payment reconciliation** — Confirming payments with Softpay and ePay
- **Terminal configuration** — Distributing settings (language, currency, machine name) to all terminals

Softpay Integration

A mobile payment solution that:

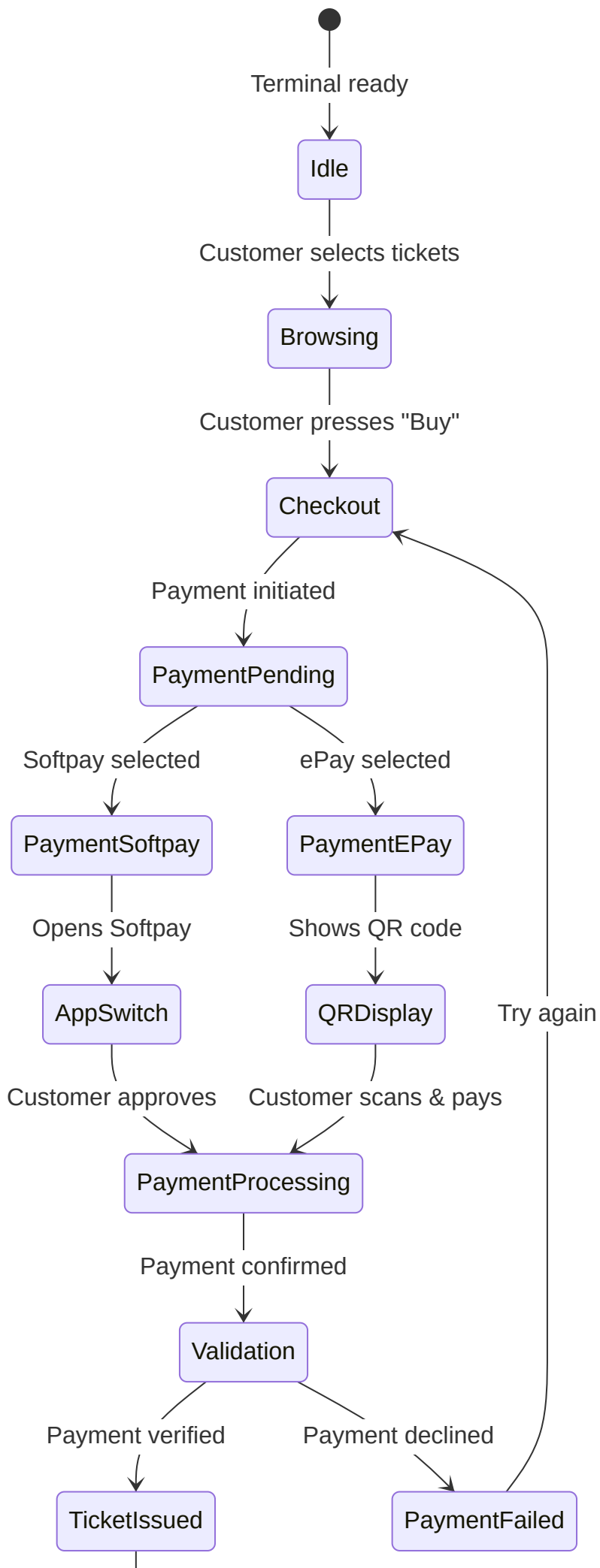
- Creates a pending transaction in the Softpay cloud
- Switches the customer to the Softpay app on the same device
- Returns to the ATVM automatically after payment approval
- Confirms the transaction through the backend

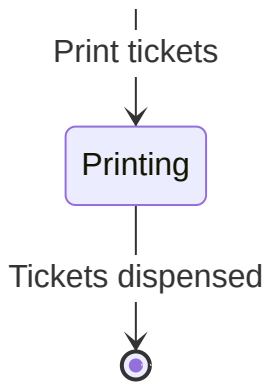
Printing System

Integrated thermal printing that produces:

- **Individual tickets** — Each with a unique ID, validity period, and scannable QR code
 - **Transaction receipt** — Summary of all purchased items, total, date, and terminal ID
-

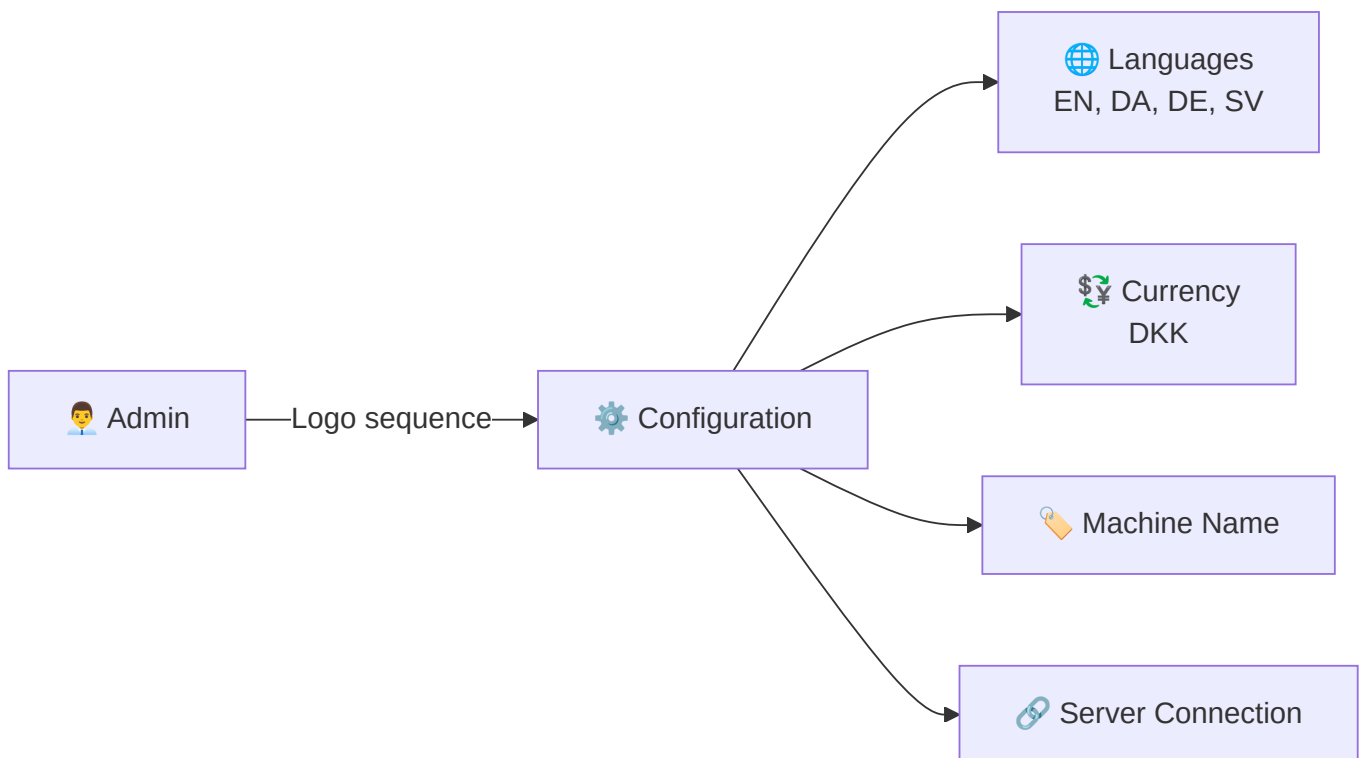
6. Transaction Flow





7. Administration & Configuration

Terminal operators can configure the system through a protected admin panel:



Configuration is set by entering a hidden sequence (tapping the logo in a specific pattern), which opens the admin panel where operators can:

- Set the terminal language
 - Configure the server connection
 - View terminal health status
 - Manage printer settings
-

8. Monitoring & Resilience

The system is designed for unattended operation with built-in resilience:

Feature	Description
Health Monitor	Continuously checks printer status and server connectivity
Error Recovery	Automatic retry mechanisms for lost server connections
Transaction Timeout	Pending transactions automatically timeout if not completed
Cart Protection	Shopping cart clears after inactivity to prevent stale orders
Printer Diagnostics	Real-time printer status (paper, temperature, cover, cutter)

9. Technology Stack

Layer	Technology
Frontend	React + TypeScript (Progressive Web App)
Build	Vite
Printer	Sunmi JS SDK (WebSocket bridge)
Payments	Softpay SDK (app switch) + ePay (QR codes)
Backend	Custom API server with MongoDB
Hosting	Self-hosted on VPS

10. Summary

The ATVM system transforms a standard Sunmi Android device into a fully autonomous ticket vending terminal. By combining a modern web-based frontend with native hardware integration (printing) and multiple payment options (Softpay and ePay), it provides passengers with a familiar, efficient self-service experience — while giving operators full remote control and visibility through a centralized backend system.

The architecture is designed for **reliability** (automatic recovery, health monitoring), **flexibility** (multi-language, configurable settings), and **simplicity** (two payment options, intuitive interface), making it suitable for deployment in ferry terminals, bus stations, and other transit environments where staffed ticket counters are impractical or too costly.